Système et Réseaux : TP10 Quizz Protocol

avril 2025 — Pierre Rousselin

Ce TP (qui devrait occuper plusieurs séances) est tiré du partiel 2021–2022.

Exercice 1 : Client et serveur « *Quizz Protocol* »

Le but de cet exercice est de créer pas-à-pas un client et un serveur implémentant le *Quizz Protocol* décrit plus loin.

- Le *Quizz Protocol* est au-dessus de TCP et est associé au port 6666.
- Le serveur dispose d'une banque de question et lorsqu'un client s'y connecte, le serveur lui envoie une question aléatoire sous la forme d'un énoncé et de plusieurs propositions.
- Le client reçoit l'énoncé de la question et les propositions et les affiche sur le terminal pour l'utilisateur.
- L'utilisateur répond par un numéro $(1, 2, ...)$ de proposition qui est ensuite transmis au serveur (éventuellement modifié de façon à ce qu'il soit compris par le serveur).
- Le serveur, ayant reçu la réponse du client, lui envoie 0 si la réponse est incorrecte ou 1 si c'est la bonne réponse.

Voici deux exemples de déroulement, côté client, en supposant que `127.0.0.1` soit bien l'adresse d'un serveur *Quizz Protocol* :

```
$ quizz/client_quizz 127.0.0.1
9a:1b:c2:70:05:9f peut être une adresse :
1. MAC
2. IPv4
3. IPv6
4. Aucune des autres réponses
5. postale
6. email
Votre réponse : 1
Bravo, c'est la bonne réponse !
$ quizz/client_quizz 127.0.0.1
Que signifie UDP ?
1. Unspecified Data Protocol
2. Union des Démocrates Populaires
3. User Datagram Protocol
4. Unordered Distribution Protocol
Votre réponse : 2
Désolé, ce n'est pas la bonne réponse.
```

Les fonctions `main` du client et du serveur sont les suivantes :

```c
/* client_quizz.c */
int main(int argc, char *argv[])
{
        if (argc != 2) {
                usage(argv[0]);
                return 1;
        }
        int sock = connection_au_serveur(argv[1]);
        echanger_avec_serveur(sock);
        close(sock);
        return 0;
}

/* serveur_quizz.c */
int main(void)
```

```
{
        init_banque_questions();
        int sock = init_sock_ecoute();
        for (;;) {
                int se = connection_au_client(sock);
                if (se < 0)
                        continue;
                echanger_avec_client(se, question_aleatoire());
                close(se);
        }
        /* On n'y arrive jamais normalement. */
        close(sock);
        detruire_banque_questions();
        return 0;
}
```

Avant de détailler plus précisément le protocole applicatif, nous commençons par l'établissement de la connection.

0. Préliminaire : Télécharger le dossier associé à ce TP sur *moodle*. Lire les prototypes et spécifications des fonctions du fichier (maison, non standard) `sys_utils.h` écrit en annexe. Vous pouvez utiliser les fonctions de `sys_utils.h` dans tout l'exercice pour gagner du temps (mais ce n'est pas une obligation). *La fonction* `exact_read` *était à implémenter lors du partiel 2021 – 2022, alors assurez-vous que vous la comprenez bien !*

1. Écrire la définition de la fonction suivante dans le fichier `client_quizz.c`.

```
/** Essayer de se connecter en TCP à l'adresse IPv4 donnée en argument, au port
 * associé au Quizz Protocol.
 * En cas de succès, retourne le descripteur de fichier associé à la socket
 * d'échange avec le serveur.
 * En cas d'échec, met fin au programme après avoir affiché un message d'erreur
 * sur la sortie standard des erreurs. */
int connection_au_serveur(const char *addr_ipv4);
```

2. Écrire la définition de la fonction suivante dans le fichier `serveur_quizz.c`

```
/** Créer et configure une socket d'écoute TCP sur IPv4 sur le port associé au
 * Quizz Protocol. Retourne le descripteur de fichier de la socket ainsi créée
 * en cas de succès, met fin au programme sinon. */
int init_sock_ecoute();
```

3. Écrire la définition de la fonction suivante :

```
/** Accepter les demandes de connection entrantes sur la socket d'écoute
 * sock.
 * Retourne le descripteur de fichier de la socket d'échange créée ou -1 en cas
 * d'erreur */
int connection_au_client(int sock);
```

Bonus : Faire en sorte que le serveur affiche sur le terminal l'adresse IP du client auquel il est connecté (utiliser les paramètres de l'appel système `accept`, la fonction `inet_ntop` et la constante symbolique `INET_ADDRSTRLEN`).

4. Juste pour tester, avant de passer aux choses sérieuses, définir les fonctions `echanger_avec_client` et `echanger_avec_serveur` de façon à ce que :

- Une fois la connection établie, le client envoie `"plop !"` (terminée par un octet nul) au serveur.
- Le serveur, après avoir reçu ce message (on suppose qu'il connaît son nombre d'octets), l'écrit sur sa sortie standard et répond `"hop !"` (terminée par un octet nul) au client.
- Après avoir reçu le message du serveur (dont le nombre d'octets est suppusé connu à l'avance) et l'avoir écrit sur sa sortie standard, le client répond `"Lol !"` (terminée par un octet nul) puis ferme sa connexion.

— Le serveur reçoit ce message (encore une fois en connaissant son nombre d'octets), l'écrit sur sa sortie standard et ferme sa connexion à son tour.

Avant de passer à la suite, il faut détailler le protocole et les données. Le type suivant est défini dans le fichier `question.h` :

```
struct question {
        char *enonce;
        char **propositions;
        uint16_t nb_props;
        uint16_t ind_rep;
        /* ... */
};
```

Le champ `enonce` contient l'adresse d'une chaîne de caractère terminée par `'\0'` qui est l'énoncé de la question. Le champ `propositions` contient l'adresse d'un tableau de propositions, chaque proposition étant l'adresse d'une chaîne de caractère terminée par `'\0'`. Les champs `nb_props` et `ind_rep` sont, respectivement, le nombre de propositions dans le tableau et l'indice de la bonne réponse dans le tableau (en commençant la numérotation à 0).

Le serveur dispose d'une banque de questions fixée, initialisée au démarrage du serveur. Le type de cette banque défini aussi dans `question.h` est

```
struct banque_questions {
        struct question **tab; /* tableau de nb_questions questions */
        int nb_questions;
};
```

Les fonctions suivantes sont déjà écrites dans `question.c` et vous n'avez pas à vous en soucier [1] :

```
/** Créer et initialiser une banque de question avec les questions du fichier
 * fichier_question.
 * Retourne l'adresse de la banque de questions ainsi créée (en cas d'erreur,
 * met fin au programme). */
struct banque_questions *init_banque_questions(const char *fichier_questions);

/** Retourner une question choisie uniformément au hasard dans la banque de
 * questions d'adresse bq */
struct question *question_aleatoire(const struct banque_questions *bq);

/** Libèrer toute la mémoire associée à la banque de questions d'adresse bq */
void detruire_banque_questions(struct banque_questions *bq);
```

Ensuite, l'échange se produit de la façon suivante :

— Le serveur commence par envoyer le nombre de propositions sur 2 octets, en *Network Byte Order*.

— Puis il envoie, à la suite, l'énoncé (avec son `'\0'` final) et les propositions, dans l'ordre, chacune avec son `'\0'` final.

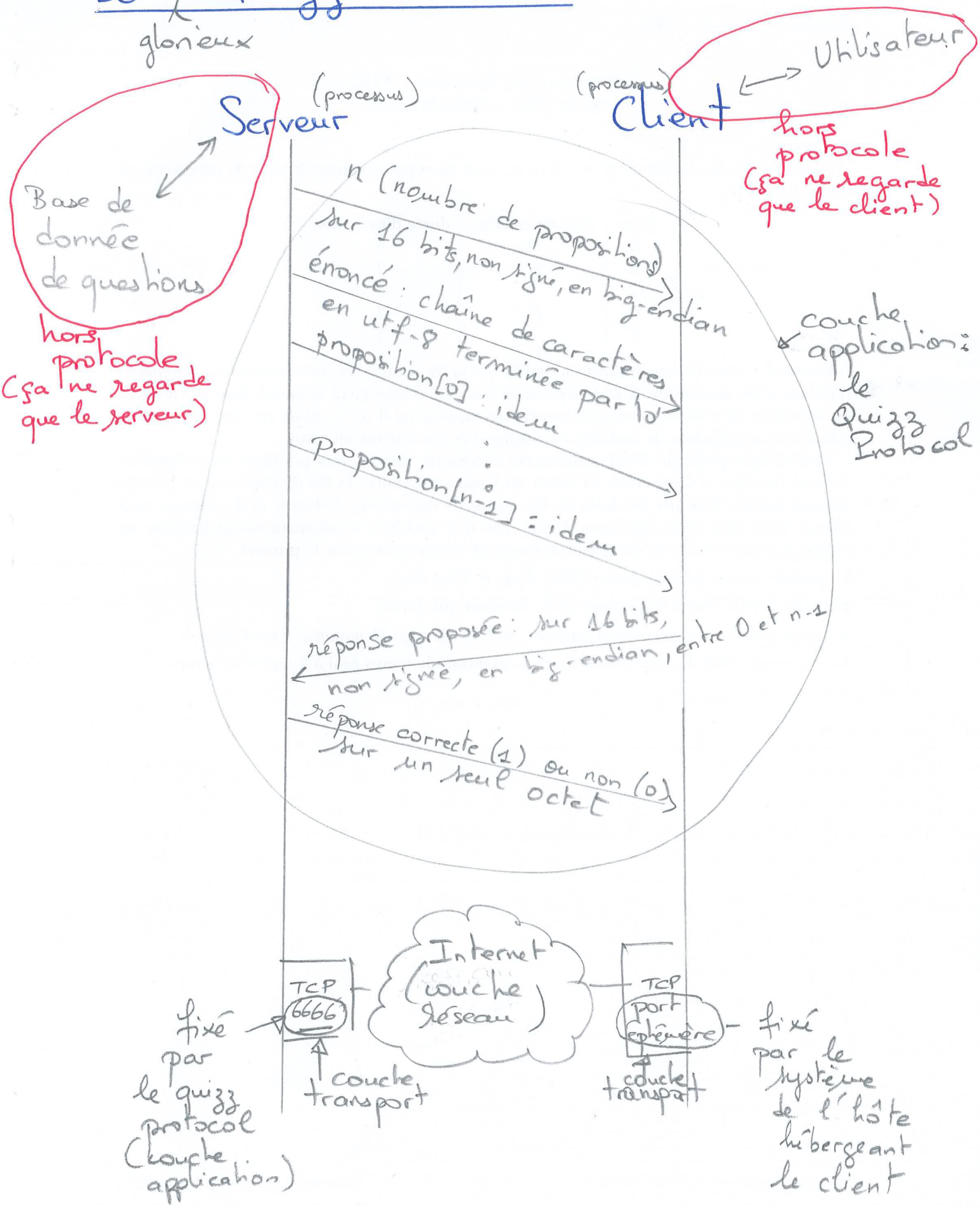
— Le client lit toutes ces informations, affiche énoncé et propositions numérotées sur le terminal (en commençant la numérotation à 1) puis attend la réponse de l'utilisateur. Enfin, il transmet cette réponse moins un, sur deux octets, en *Network Byte Order* au serveur.

— Le serveur lit la réponse de l'utilisateur et la compare à la bonne réponse. Il envoie enfin un seul octet au client, égal à 0 si la réponse est incorrecte et à 1 si c'est la bonne réponse.

Le schéma de la page suivante résume ce protocole applicatif très simple.

1. Conseil : lire et comprendre ces fonctions chez vous.

Le glorieux Quizz Protocol



5. a) Combien de propositions différentes peut-il y avoir au maximum dans une question ?

b) Qu'est-ce que le *Network Byte Order* ?

c) À quoi sert le caractère `'\0'` à la fin de l'énoncé de la question ?

d) Pourquoi faut-il envoyer le nombre de propositions ?

6. Écrire la définition de la fonction (côté serveur) :

```
/* Mener tous les échanges avec le client connecté à la socket fd, ainsi que
 * spécifié dans le Quizz Protocol, avec la question d'adresse q */
void echanger_avec_client(int fd, const struct question *q);
```

qui mène tous les échanges décrits précédemment, côté serveur, étant donnés le descripteur de fichier `fd` d'une socket d'échange et l'addresse `q` d'une question.

7. Écrire la définition de la fonction (côté client) :

```
/** Échanger avec le serveur connecté via la socket sock ainsi que spécifié par
 * le protocole Quizz Protocol, tout en menant les échanges avec l'utilisateur.
 *
 * - Imprimer sur la sortie standard la question et les propositions envoyées
 *   par le serveur
 * - Demander à l'utilisateur un entier entre 1 et le nombre de propositions
 * - Envoyer le numéro de la réponse choisie au serveur
 * - Recevoir le message final du serveur (réponse correcte ou incorrecte)
 * - Dire à l'utilisateur si sa réponse est correcte ou non
 */
void echanger_avec_serveur(int sock);
```

qui mène tous les échanges décrits précédemment, côté client, étant donnés le descripteur de fichier `fd` d'une socket d'échange connectée avec le serveur.

Il faut que chaque proposition soit convenablement numérotée (en partant de 1).

Vous pourrez utiliser la fonction suivante (définie dans `sys_utils.c`) :

```
/* Retourner un entier positif saisi par l'utilisateur, sur deux octets, compris
 * entre borne_inf (incluse) et borne_sup (incluse) */
uint16_t saisir_entre_bornes(uint16_t borne_inf, uint16_t borne_sup);
```

Indication : pour les lectures de chaînes de caractère, il est sans doute plus simple de lire 1 seul caractère à la fois et de l'afficher immédiatement.

8. a) Que se passe-t-il si un deuxième client cherche à se connecter au serveur lorsque l'utilisateur d'un premier client est en train de réfléchir à sa réponse (pendant peut-être longtemps) ? Dans quels appels systèmes sont bloqués le serveur et les deux clients ?

b) On considère le type structuré suivant :

```
struct work {
        int socket;
        struct question *q;
};
```

Écrire la définition de la fonction suivante :

```
/* Reçoit l'adresse d'une variable de type struct work dynamiquement allouée sur
 * le tas et convenablement initialisée, puis échange avec un client en
 * utilisant le champ socket de cette structure pour la question (champ q) de
 * cette structure.
 * À la fin des échanges, libère la mémoire associée à l'adresse work.
 * Retourne toujours NULL. */
void *worker(void *work);
```

c) Écrire tous les changements dans la fonction `main` du serveur pour que chaque client soit servi par un *thread* séparé. Faire en sorte que la mémoire occupée par ce *thread* soit libérée lorsqu'il se termine.

--- * ---

Fonctions utilitaires pour gagner du temps : `sys_utils.h`

```
/** Allouer nbytes octets et renvoie un pointeur sur la mémoire allouée.
 * En cas d'échec, affiche un message d'erreur et met fin au programme avec
 * statut de sortie 2. */
void *malloc_or_exit(size_t nbytes);

/** Lire exactement nbytes octets (si besoin en appelant plusieurs fois read)
 * dans le ficher de descripteur fd ou bien jusqu'à la fin du fichier et les
 * copier dans buf.
 * Retourne le nombre d'octets lus ou -1 en cas d'erreur. */
ssize_t exact_read(int fd, void *buf, size_t nbytes);

/** Utiliser la fonction perror avec l'argument err_mess et mettre fin au
 * programme avec statut de sortie 2. */
void perror_exit(const char *err_mess);

/** Vider le tampon associé à l'entrée standard */
void vider_buffer();

/** Retourner un entier positif saisi par l'utilisateur, sur deux octets, compris
 * entre borne_inf (incluse) et borne_sup (incluse) */
uint16_t saisir_entre_bornes(uint16_t borne_inf, uint16_t borne_sup);
```

Prototypes des fonctions à utiliser, quelques noms de constantes symboliques et quelques champs de types structurés

Certains arguments sont mis à `NULL` ou `0` car nous n'avons pas vu leur rôle en cours.

```
int getchar();
int putchar(int c);

size_t strlen(const char *s);
char *strcpy(char *dest, const char *src);
void *memcpy(void *dest, const void *src, size_t n);
void *memset(void *s, int c, size_t n);

pid_t fork();
pid_t wait(int *wstatus);
void exit(int status);

int pthread_create(pthread_t *thread, NULL,
                       void *(*start_routine) (void *), void *arg);
int pthread_join(pthread_t thread, void **retval);
int pthread_detach(pthread_t thread);

int pthread_mutex_init(pthread_mutex_t *mutex, NULL);
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
int pthread_mutex_destroy(pthread_mutex_t *mutex);

int sem_init(sem_t *sem, 0, unsigned int valeur);
int sem_wait(sem_t *sem);
int sem_post(sem_t *sem);
int sem_destroy(sem_t *sem);
```

```c
int open(const char *path, int oflag, ...);
ssize_t read(int fildes, void *buf, size_t nbyte);
ssize_t write(int fildes, const void *buf, size_t nbyte);
int close(int fildes);

int socket(int domain, int type, 0);
int bind(int socket, const struct sockaddr *address, socklen_t address_len);
int listen(int socket, int backlog);
int accept(int socket, struct sockaddr *address, socklen_t *address_len);
int connect(int socket, const struct sockaddr *address, socklen_t address_len);

ssize_t recv(int socket, void *buffer, size_t length, int flags);
ssize_t send(int socket, const void *buffer, size_t length, int flags);
ssize_t recvfrom(int socket, void *buffer, size_t length, int flags,
                 struct sockaddr *address, socklen_t *address_len);
ssize_t sendto(int socket, const void *message, size_t length, int flags,
               const struct sockaddr *dest_addr, socklen_t dest_len);


uint32_t htonl(uint32_t hostlong);
uint16_t htons(uint16_t hostshort);
uint32_t ntohl(uint32_t netlong);
uint16_t ntohs(uint16_t netshort);

const char *inet_ntop(int af, const void *src, char *dst, socklen_t size);
int inet_pton(int af, const char *src, void *dst);

struct sockaddr_in {
        sa_family_t    sin_family; /* address family: AF_INET */
        in_port_t      sin_port;   /* port in network byte order */
        struct in_addr sin_addr;   /* internet address */
};
struct in_addr {
        in_addr_t s_addr;
};
```

Quelques constantes symboliques :

- `AF_UNIX`, `AF_INET`, `AF_INET6`
- `SOCK_DGRAM`, `SOCK_RAW`, `SOCK_STREAM`
- `INADDR_ANY`, `IN6ADDR_ANY_INIT`
- `INET_ADDRSTRLEN`, `INET6_ADDRSTRLEN`
- `O_RDONLY`, `O_WRONLY`, `O_RDWR`, `O_CREAT`, `O_APPEND`, `O_TRUNC`